

INTERNAL DOCUMENT

Multi-Agent AI Day Planner

Technical Specification & Project Definition

FastAPI	LangGraph	OpenAI gpt-4o	Groq llama-3.1-8b	Claude (Anthropic)
Streamlit	Pinecone	Redis 7	Prometheus/Grafana	Azure Container Apps

June 2026 · Version 1.1 · 14 Pages

1

Overview

Project type · Problem statement · Target users · Tech stack

1.1 Project Type

Dimension	Value
Project category	Multi-agent AI system for personalised day planning
Primary interface	Streamlit web UI + REST API (FastAPI)
Deployment target	Azure Container Apps (Bicep IaC + GitHub Actions CI/CD)
Cost model	Cost-aware agent routing — sub-\$0.01 per user per day
Agents	OpenAI gpt-4o (paid) · Groq llama-3.1-8b-instant (free) · Claude (local-equivalent zero-marginal-cost fallback)

1.2 Problem Statement

Professionals lack a unified AI system that intelligently plans their entire day — from wake-up to sleep — incorporating personalised diet plans, structured workout and fitness sessions, and goal-aware scheduling. Existing wellness tools are either too generic to be actionable, prohibitively expensive, or require stitching together multiple disconnected applications with no unified intelligence layer. There is no single system that orchestrates planning, nutrition, and fitness through a cost-aware multi-agent architecture.

1.3 Target Users

User segment	Primary need
Working professionals	Structured daily planning across health, nutrition, and fitness
Fitness-conscious individuals	Weight management, macro tracking, and consistent workout scheduling

1.4 Tech Stack

Layer	Technologies
Backend & AI	FastAPI · LangGraph · OpenAI gpt-4o · Groq llama-3.1-8b-instant · Claude (Anthropic)
Frontend	Streamlit — 4-page web UI with sidebar navigation (Day Planner, History, Feedback, Analytics)
Vector store	Pinecone (Azure eastus, 1536 dims, cosine) · FAISS local fallback
State & persistence	SQLite · Redis 7 (cache / history layer)
Observability	Prometheus · Grafana 10.2 · Loki 2.9 · Promtail
Infrastructure	Docker Compose · Azure Container Apps · ACR · Bicep IaC
CI/CD	GitHub Actions — test → build → push ACR → deploy Bicep on every push to main

2

Goals

8 architectural goals with precise technical outcomes

G1

Cost-aware multi-agent orchestration via LangGraph

A LangGraph state machine with five nodes routes each request to the cheapest capable agent via a static routing table mapping 10 task types to OpenAI, Groq, or Claude. Unknown task types default to the lowest-cost fallback agent, Claude. Every agent call increments a Prometheus counter and records latency in a histogram. Measurable: OpenAI share of total agent calls $\leq 30\%$.

G2

Personalised daily schedule generation

Generates a full wake-to-sleep timeline from user-configured wake-up time, sleep time, diet preference (Veg/Non-Veg), fitness preference (Gym/Yoga/Both), workout duration, and freeform extra preferences. Each event carries: time, activity, category, duration_minutes, and notes. Fixed rules enforced in the prompt: workout block sized to the selected workout_duration, meals matched to diet preference, and an evening wind-down block before sleep. Measurable: p95 generation latency < 10 seconds.

G3

Meal planning aligned to diet preference

Generates Indian meals for breakfast, lunch, and dinner aligned to the user's selected Diet Preference (Veg / Non-Veg) captured at plan-generation time. Each meal includes name, estimated calories, and ingredient guidance. Macros balanced and cuisine varied across the week. Measurable: meal block present in every generated day plan response.

G4

Fitness scheduling aligned to fitness preference

Workout block generated according to the user's selected Fitness Preference (Gym / Yoga / Both) and Workout Duration (e.g. 1 hr) captured in the Day Planner form. The prompt enforces the exact selected duration and fitness mode rather than a generic placeholder block. Measurable: workout event duration_minutes always matches the user-selected Workout Duration field.

G5

Extra-preferences free-text steering

The Extra Preferences field (e.g. 'office 9 to 6, avoid rice at night, evening gym') is passed verbatim into the Agent 1 prompt as an additional constraint layer on top of the structured fields. The model is instructed to honour these constraints without contradicting the structured diet/fitness selections. Measurable: generated plan reflects free-text constraints when present.

G6

Disease and disability aware safety guardrails

Agent 3 reads the user's stored diseases and disability fields from signup and generates rule-based safety notes (e.g. low-salt guidance for BP, avoid intense workout for heart conditions). These notes are injected into the Agent 1 prompt so the generated plan never contradicts a known health constraint. Measurable: safety notes present in 100 % of plans where diseases or disability is non-empty.

G7

Plan history and feedback retention

Every generated day plan is automatically saved before returning the response, scoped per user, and retrievable through the History page with a configurable limit. The Feedback page lets the user rate and comment on the most recently generated plan, linked by plan ID. Measurable: plan and feedback rows are queryable immediately after generation and submission respectively.

3

Non-Goals

6 deliberate architectural decisions with technical reasoning

NG1**No wearable / biometric integration**

Smartwatch or fitness-tracker APIs require per-user OAuth2 credential storage, real-time event streaming infrastructure, and a secrets management layer that conflicts with the stateless container scaling model. Deferred to v2 as a separate microservice.

NG2**No native mobile application**

Streamlit serves via a web server and is not compiled to a mobile binary. A native app would require a separate build pipeline, app-store deployments, and push-notification infrastructure — out of scope for v1, which targets the browser-based experience exclusively.

NG3**No multi-user social or collaboration features**

All history is keyed on a per-user identity with no social graph layer. Adding friend graphs or leaderboards would require additional relational schema and privacy controls — orthogonal to the AI orchestration architecture in scope for v1.

NG4**No payment processing or subscription tiers**

The cost model targets sub-\$0.01 per user per day through cost-aware routing. Adding billing would require Stripe integration, subscription state management, and feature-flag gating — deferred in favour of demonstrating the core multi-agent orchestration capabilities cleanly.

NG5**No medical diagnosis or clinical decision support**

All AI agents are positioned as a wellness coach, not a physician. Safety notes generated from disease and disability fields are advisory only. Clinical output would require regulatory compliance and clinical validation — explicitly out of scope.

NG6**No SMS or push notification reminders**

The application deliberately omits any SMS gateway, scheduler job store, or push notification infrastructure. Reminders are not part of the v1 scope — the product surface is limited to plan generation, history, and feedback within the Streamlit session.

4

Success Metrics

Quantitative targets with measurement instrumentation

Metric	Target	Measurement method
Plan generation latency p95	< 10 seconds	http_request_duration_seconds{path='/generate'} – p95 bucket
Cost per user per day	< \$0.01	agent_calls_total{agent='openai'} × token cost, from usage logs
API uptime (Azure)	> 99.5 %	Prometheus Blackbox Exporter on GET /health
API request error rate	< 1 %	api_requests_total{status=~'5..'} / api_requests_total
OpenAI share of agent calls	≤ 30 %	agent_calls_total{agent='openai'} / agent_calls_total
Feedback submission rate	> 40 % of generated plans	COUNT(feedback) / COUNT(day_plans) per rolling 7-day window
Average plan rating	> 3.5 / 5	AVG(feedback.rating) – SQLite aggregate query

5

Feature Definitions

Features aligned to the live Streamlit UI — implementation detail, data flow, agent ownership

5.1

Orchestrator

Agent/Component: LangGraph — routes to OpenAI · Groq · Claude

Architecture	LangGraph state machine with five nodes. State object carries: task_type, user_input, context, agent_result.
Routing table	10 task types mapped statically: schedule/meals → OpenAI; quick lookups/analysis → Groq; summarise/evaluate/fallback → Claude. Unknown type → Claude (lowest marginal cost).
Agent nodes	OpenAI: high-quality generation (day plan, meals). Groq: speed-critical lightweight tasks. Claude: summarisation, evaluation, and default fallback for unrecognised task types.
Observability	Every agent call increments agent_calls_total{agent} counter. Latency recorded in agent_latency_seconds{agent} histogram. Middleware records every API request: method, path, status, elapsed — shipped to Loki.

5.2

Day Planner (main screen)

Agent/Component: OpenAI gpt-4o

UI fields	Wake-up time · Sleep time · Diet Preference (Veg/Non-Veg) · Fitness Preference (Gym/Yoga/Both) · Workout Duration (e.g. 1 hr) · Extra Preferences (freeform textarea) · optional Phone field in the sidebar.
Endpoint	POST /planner/generate — accepts wake_time, sleep_time, diet_type, fitness_type, workout_duration, preferences (freeform string), user_id.
Prompt rules	Workout block matches the selected Workout Duration and Fitness Preference exactly · meals matched to Diet Preference · Extra Preferences honoured as additional constraints without contradicting the structured fields.
Event schema	time (HH:MM) · activity (str) · category (wake meal work break workout reading sleep) · duration_minutes (int) · notes (str).
Response	{plan_id, date, user_id, events: [...], agent_analysis: {agent1, agent2, agent3}} — plan auto-saved to history before return.

5.3

Authentication & Session

Agent/Component: auth_service — email + password

UI element	Sidebar header shows 'Logged in as: {full name}' followed by the user's email as a mailto link, matching the live screenshot (Sraddhanjali Padhan / padhansradhanjali@gmail.com).
Signup	Captures name, email, phone, password, height, weight, gender, age, profession, diseases (multiselect), disability — used later by Agent 3 for safety notes.
Login	Email + optional password. Demo mode allows email-only login; production should always require password.
Session state	st.session_state stores logged_in, user (dict), auth_page, and latest_plan_id for gating the Feedback page.

5.4

History

Agent/Component: SQLite — no agent

UI element	Sidebar radio item '■ History' — second item in the navigation list, matching the live screenshot.
------------	--

Endpoint	GET /history/{user_id}?limit=N — returns up to N most recent plans, descending chronological order.
Data shown	Plan date, wake/sleep time, diet/fitness preference, full event list, and the Agent 2/Agent 3 analysis captured at generation time.

5.5

Feedback
Agent/Component: SQLite — no agent

UI element	Sidebar radio item '■ Feedback' — third item in the navigation list, matching the live screenshot.
Endpoint	POST /feedback — accepts user_id, plan_id, rating (1–5), comments.
UI guard	Form only renders when st.session_state.latest_plan_id is set, i.e. after a plan has been generated in the current session.

5.6

Analytics
Agent/Component: Prometheus / Grafana — read-only stub

UI element	Sidebar radio item '■ Analytics' — fourth item in the navigation list, matching the live screenshot.
Behaviour	Displays links to the FastAPI Swagger docs and the /health endpoint for now. Designed as the future home of the Grafana dashboard panels described in Section 5.8 of observability.

5.7

Observability

Agent/Component: Prometheus · Grafana 10.2 · Loki 2.9 · Promtail

Prometheus metrics	agent_calls_total{agent} counter · agent_latency_seconds{agent} histogram · api_requests_total{method,endpoint,status} counter · active_users gauge.
Log shipping	Custom Python handler batches log entries every 2 seconds → structured JSON to Grafana Loki via HTTP. Bounded in-memory queue — emit calls never block the async event loop.
Grafana dashboard	Panels provisioned at container startup: agent calls/min by agent · agent latency p95 by agent · API requests/min by endpoint · live Loki log stream filtered to application label.
Log collection	Promtail scrapes stdout from all Docker containers via the Docker socket, applies container/service labels, pushes to Loki every 5 seconds.

6

UI Mock

Streamlit application — sidebar navigation matched to live screenshot

The Streamlit application's Day Planner page is shown below, matching the structure of the captured screenshot: a sidebar with the app identity, a 'Logged in as' session block, four navigation items (Day Planner, History, Feedback, Analytics), and an optional Phone field — alongside the main panel's Wake-up time, Sleep time, Diet Preference, Fitness Preference, Workout Duration, and Extra Preferences fields.

● ● ● localhost:8501 · AI Day Planner

■ AI Day Planner

Multi-Agent · Indian Meals · SMS
Optional

Logged in as: **Sraddhanjali Padhan**

padhansradhanjali@gmail.com

● ■ Day Planner

■ ■ History

■ ■ Feedback

■ ■ Analytics

Phone Optional

Wake-up time

21:57

Sleep time

21:57

Diet Preference

Veg

Fitness Preference

Gym

Workout Duration

1 hr

Extra Preferences

Example: office 9 to 6, avoid rice at night, evening gym

Figure 1 — Streamlit Day Planner UI, reconstructed from the captured screenshot (30 June 2026)

7

Azure Deployment

Bicep IaC · GitHub Actions CI/CD · Azure Container Apps

The entire infrastructure is defined as code using Azure Bicep (**azure/main.bicep**) and deployed via a single GitHub Actions workflow on every push to the main branch. The workflow runs unit tests first, then provisions infrastructure, builds both Docker images in Azure Container Registry, and deploys them to Container Apps.

7.1 Infrastructure Summary

Azure resource	Purpose / configuration
Azure Container Registry	Stores the FastAPI and Streamlit Docker images built by the CI pipeline.
Azure Cache for Redis	Hosts plan/feedback history cache layer. TLS-only, Basic C0 tier.
Log Analytics Workspace	Receives container logs from the Container Apps environment. 30-day retention.
Container Apps Environment	Shared managed environment for both application containers.
FastAPI Container App	Runs the API — 0.5 vCPU, 1 GiB RAM, external HTTPS ingress, scales 1–5 replicas.
Streamlit Container App	Runs the UI — 0.25 vCPU, 0.5 GiB RAM, external HTTPS ingress, scales 1–3 replicas.

7.2 Secrets & Configuration

API keys (OpenAI, Groq, Anthropic) are passed as secure Bicep parameters and stored as Container App secrets. They are never written to ARM output or environment files. The Redis connection string is constructed dynamically at deploy time from the Redis primary key and stored the same way. Each container references secrets by name rather than injecting values directly into environment variables.

7.3 CI/CD Pipeline

Step	Action	Detail
Trigger	Push to main branch	Every commit to main triggers the full pipeline.
Step 1	Unit tests	Pipeline stops on failure — no build or deploy on test failure.
Step 2	Azure login + Bicep	Infrastructure provisioned via <code>azure/main.bicep</code> — idempotent.
Step 3	Build FastAPI image	Docker build → push to ACR, tagged with the Git commit SHA.
Step 4	Build Streamlit image	Docker build → push to ACR, tagged with the Git commit SHA.

8

Contributors

Project team

SP

Sraddhanjali Padhan

Project Contributor

VK

Vishal Kumar

Project Contributor

This technical specification and the accompanying multi-agent day planner system were jointly authored and reviewed by the contributors listed above.

Document	Version	Date	Classification	Pages
Multi-Agent AI Day Planner — Technical Specification	1.1	June 2026	Internal	14